МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет по лабораторной работе №6

Выполнила:

Студент группы БВТ2504
Мясникова Е.А.

Проверила: Шедова А.Р

Москва, 2026 г.

Содержание

Цель работы:.....2
Ход выполненной работы:.....2
1. Задание 1.....2
2. Задание 2.....4
3. Задание 3.....5
4. Задание 4.....11
5. Задание 5.....12
Заключение.....13

Цель работы: Научиться реализовывать алгоритмы поиска в отсортированных линейных структурах и экспериментально определять область эффективности каждого из них в зависимости от распределения данных.

Ход выполненной работы:

1. **Задание 1**

Реализованы линейный, бинарный и интерполяционный поиск в отсортированном массиве. Каждая функция возвращает индекс найденного элемента или -1 и количество выполненных сравнений. В бинарном поиске поддерживается инвариант границ, а в интерполяционном предусмотрены проверка диапазона ключа и защита от деления на ноль.

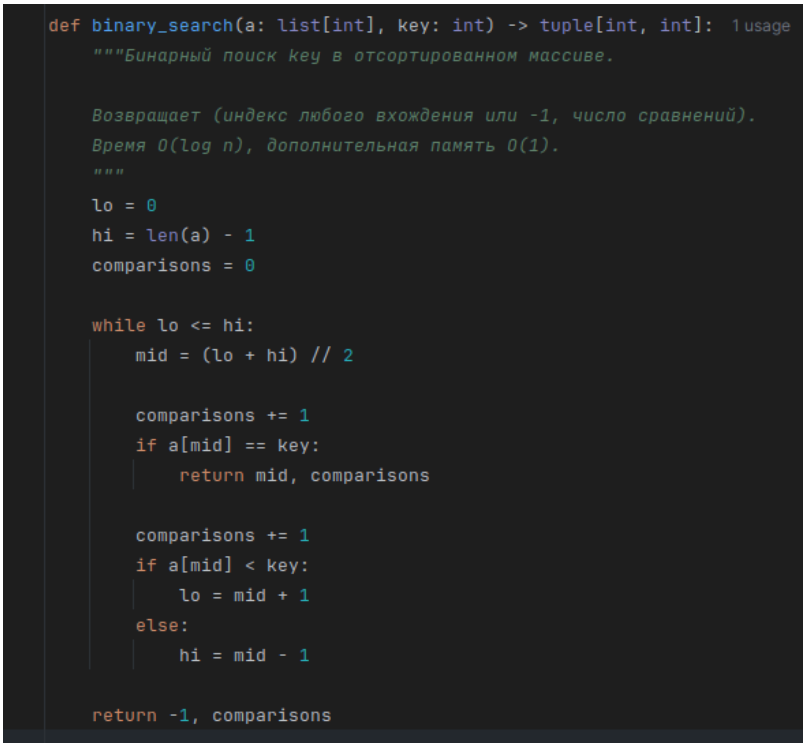
```python
def linear_search(a: list[int], key: int) -> tuple[int, int]:  2 usages
    """Линейный поиск первого вхождения key.

    Возвращает (индекс первого вхождения или -1, число сравнений).
    Время O(n), дополнительная память O(1).
    """
    comparisons = 0

    for index, value in enumerate(a):
        comparisons += 1
        if value == key:
            return index, comparisons

    return -1, comparisons
```

Рисунок 1 - Реализация линейного поиска со счётчиком сравнений



```python
def binary_search(a: list[int], key: int) -> tuple[int, int]:  1 usage
    """Бинарный поиск key в отсортированном массиве.

    Возвращает (индекс любого вхождения или -1, число сравнений).
    Время O(log n), дополнительная память O(1).
    """
    lo = 0
    hi = len(a) - 1
    comparisons = 0

    while lo <= hi:
        mid = (lo + hi) // 2

        comparisons += 1
        if a[mid] == key:
            return mid, comparisons

        comparisons += 1
        if a[mid] < key:
            lo = mid + 1
        else:
            hi = mid - 1

    return -1, comparisons
```

Рисунок 2 - Реализация бинарного поиска с изменением границ поиска

```python
def interpolation_search(a: list[int], key: int) -> tuple[int, int]:  1 usage
    """Интерполяционный поиск key в отсортированном массиве.

    Возвращает (индекс любого вхождения или -1, число сравнений).
    Среднее время на равномерных данных O(log log n), худшее O(n).
    Дополнительная память O(1).
    """
    lo = 0
    hi = len(a) - 1
    comparisons = 0

    while lo <= hi:
        comparisons += 1
        if a[lo] == a[hi]:
            comparisons += 1
            if a[lo] == key:
                return lo, comparisons
            return -1, comparisons

        comparisons += 1
        if key < a[lo]:
            return -1, comparisons

        comparisons += 1
        if key > a[hi]:
            return -1, comparisons

        mid = lo + (key - a[lo]) * (hi - lo) // (a[hi] - a[lo])

        comparisons += 1
        if a[mid] == key:
            return mid, comparisons

        comparisons += 1
        if a[mid] < key:
            lo = mid + 1
        else:
            hi = mid - 1

    return -1, comparisons
```

Рисунок 3 - Реализация интерполяционного поиска с защитой от деления на ноль

```
self_check: OK
linear: дополнительные проверки - OK
binary: дополнительные проверки - OK
interpolation: дополнительные проверки - OK
Задание №1: все проверки пройдены.
```

Рисунок 4 - Результаты проверки трёх алгоритмов поиска

Вывод: Реализованы три алгоритма поиска со счётчиками сравнений. Линейный поиск возвращает первое вхождение, бинарный и интерполяционный любое подходящее. Выполнена проверка на граничных случаях, повторяющихся значениях и случайных массивах.

2. Задание 2

Для варианта 16 сгенерированы отсортированные массивы равномерного и экспоненциального распределений. Для каждого распределения использованы пять размеров массива. Для каждого массива подготовлено по 200 присутствующих и 200

отсутствующих ключей. Проверены длина и упорядоченность массивов, количество ключей и соответствие ключей своим наборам.

```python
def make_dataset(n: int, distribution: str, rng: random.Random) -> list[int]:  1 usage
    """Отсортированный массив n ключей: 'uniform' — равномерное распределение,
    'exponential' — неравномерное (экспоненциальное, значения сгущаются у нуля)."""
    if distribution == "uniform":
        return sorted(rng.randint( a: 0, 10 * n) for _ in range(n))
    if distribution == "exponential":
        return sorted(int(rng.expovariate(1.0 / 1000.0)) for _ in range(n))
    raise ValueError(f"неизвестное распределение: {distribution!r}")


def pick_keys(a: list[int], rng: random.Random, count: int,  1 usage
              present: bool) -> list[int]:
    """count ключей для серии поисков: присутствующие берутся из массива,
    отсутствующие подбираются вне множества его значений."""
    if present:
        return [a[rng.randrange(len(a))] for _ in range(count)]
    have = set(a)
    hi = (a[-1] if a else 0) + 2
    keys: list[int] = []
    while len(keys) < count:
        k = rng.randint(-hi, hi)
        if k not in have:
            keys.append(k)
    return keys
```

Рисунок 5 - Генерация массивов и наборов поисковых ключей

```
Вариант 16, seed = 46
Распределение         n      min      max   есть    нет
uniform            1000       16     9998    200    200
uniform            3000        0    29991    200    200
uniform           10000       14    99991    200    200
uniform           30000       10   299995    200    200
uniform          100000       17   999999    200    200
exponential        1000        1     5691    200    200
exponential        3000        0     9238    200    200
exponential       10000        0    10667    200    200
exponential       30000        0     9221    200    200
exponential      100000        0    10614    200    200
Задание №2: данные сгенерированы, все проверки пройдены.
```

Рисунок 6 - Результаты генерации и проверки данных

Вывод: Подготовлены десять отсортированных массивов двух распределений и наборы поисковых ключей. Данные проверены и готовы для сравнения алгоритмов поиска.

3. Задание 3

Проведено сравнение линейного, бинарного и интерполяционного поиска на массивах пяти размеров с равномерным и экспоненциальным распределениями. Для каждого массива отдельно исследован поиск присутствующих и отсутствующих ключей. Измерены среднее число сравнений и время поиска. Перед измерениями выполнялся прогрев, каждая серия повторялась пять раз, для времени использовалась

медиана. Генерация и сортировка массивов выполнялись вне измеряемого участка. Результаты сохранены в таблицу и представлены на графиках.

```
import argparse
import bisect
import csv
import math
import platform
import random
import statistics
import time
from pathlib import Path
```

Рисунок 7 - Подключение модулей для проведения эксперимента и сохранения результатов

```
def plot_results(rows: list[dict], output_dir: Path) -> None:  new *
    """Построить графики сравнений и времени для обоих распределений."""
    import matplotlib.pyplot as plt

    plt.rcParams.update({"font.size": 12})
    styles = {
        "linear": ("Линейный", "C0"),
        "binary": ("Бинарный", "C1"),
        "interpolation": ("Интерполяционный", "C2"),
    }
    titles = {
        "uniform": "Равномерное распределение",
        "exponential": "Экспоненциальное распределение",
    }

    for distribution in titles:
        fig_c, axes_c = plt.subplots(
            nrows: 1,  ncols: 2, figsize=(11, 4.5), layout="constrained"
        )
        fig_t, axes_t = plt.subplots(
            nrows: 2,  ncols: 2, figsize=(11, 8), layout="constrained"
        )
        fig_c.suptitle(titles[distribution])
        fig_t.suptitle(titles[distribution])

        for col, present in enumerate((True, False)):
            title = "Ключ присутствует" if present else "Ключ отсутствует"

            for name, _ in ALGORITHMS:
                points = [
                    row for row in rows
                    if row["distribution"] == distribution
                    and row["present"] == present
                    and row["algorithm"] == name
                ]
                points.sort(key=lambda row: row["n"])
                sizes = [row["n"] for row in points]
                comparisons = [row["comparisons"] for row in points]
                times = [row["time_us"] for row in points]
                label, color = styles[name]
```

Рисунок 8 - Подготовка данных для построения графиков

```
                axes_c[col].loglog(
                    sizes, comparisons, "o-", label=label, color=color
                )
                axes_t[0, col].loglog(
                    sizes, times, "o-", label=label, color=color
                )
                if name != "linear":
                    axes_t[1, col].plot(
                        [math.log2(n) for n in sizes], times,
                        "o-", label=label, color=color
                    )

            axes_c[col].set_title(title)
            axes_c[col].set_xlabel("Размер массива n")
            axes_c[col].set_ylabel("Среднее число сравнений")
            axes_t[0, col].set_title(title)
            axes_t[0, col].set_xlabel("Размер массива n")
            axes_t[1, col].set_title(title + ": время от log₂(n)")
            axes_t[1, col].set_xlabel("log₂(n)")
            for row in range(2):
                axes_t[row, col].set_ylabel("Время одного поиска, мкс")

        for ax in [*axes_c, *axes_t.flat]:
            ax.grid(True, which="both", alpha=0.3)
            ax.legend(fontsize=10)

        fig_c.savefig(output_dir / f"comparisons_{distribution}.png", dpi=180)
        fig_t.savefig(output_dir / f"time_{distribution}.png", dpi=180)
        plt.close(fig_c)
        plt.close(fig_t)
```

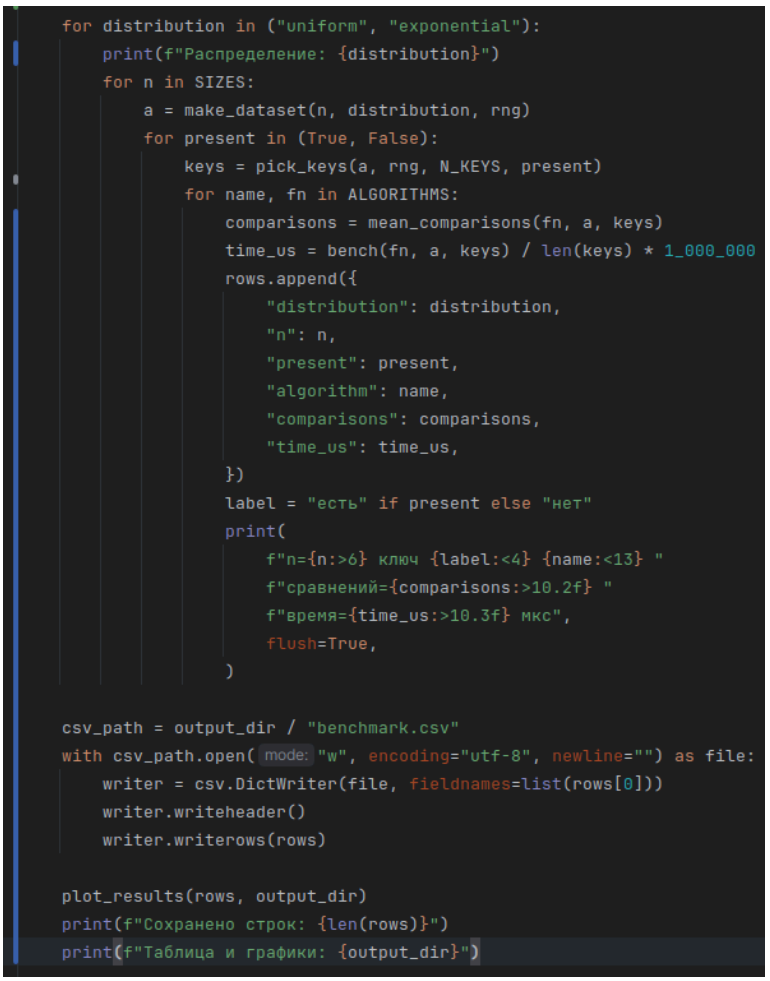
Рисунок 9 - Построение и сохранение графиков сравнений и времени поиска

```
def run_benchmarks(seed: int) -> None:  Lena *
    rng = random.Random(seed)
    rows = []
    output_dir = Path(__file__).resolve().parent / "results"
    output_dir.mkdir(exist_ok=True)

    cpu = platform.processor() or "не определён"
    cpuinfo = Path("/proc/cpuinfo")
    if cpuinfo.exists():
        for line in cpuinfo.read_text(encoding="utf-8").splitlines():
            if line.startswith("model name"):
                cpu = line.split( sep: ":",  maxsplit: 1)[1].strip()
                break

    environment = (
        f"CPU: {cpu}\nPython: {platform.python_version()}\n"
        f"Вариант: {seed - 30}\nSeed: {seed}\n"
        f"Размеры: {SIZES}\nПовторы: {REPEATS}\nКлючей в серии: {N_KEYS}\n"
        "Время: медиана времени серии / число ключей, микросекунды\n"
    )
    (output_dir / "environment.txt").write_text(environment, encoding="utf-8")
    print(environment)
```

Рисунок 10 - Подготовка эксперимента и сохранение параметров компьютера



```
for distribution in ("uniform", "exponential"):
    print(f"Распределение: {distribution}")
    for n in SIZES:
        a = make_dataset(n, distribution, rng)
        for present in (True, False):
            keys = pick_keys(a, rng, N_KEYS, present)
            for name, fn in ALGORITHMS:
                comparisons = mean_comparisons(fn, a, keys)
                time_us = bench(fn, a, keys) / len(keys) * 1_000_000
                rows.append({
                    "distribution": distribution,
                    "n": n,
                    "present": present,
                    "algorithm": name,
                    "comparisons": comparisons,
                    "time_us": time_us,
                })
                label = "есть" if present else "нет"
                print(
                    f"n={n:>6} ключ {label:<4} {name:<13} "
                    f"сравнений={comparisons:>10.2f} "
                    f"время={time_us:>10.3f} мкс",
                    flush=True,
                )

csv_path = output_dir / "benchmark.csv"
with csv_path.open( mode: "w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=list(rows[0]))
    writer.writeheader()
    writer.writerows(rows)

plot_results(rows, output_dir)
print(f"Сохранено строк: {len(rows)}")
print(f"Таблица и графики: {output_dir}")
```

Рисунок 11 - Измерение характеристик поиска и сохранение результатов

	distribution	n	present	algorithm	comparisons	time_us
1	uniform	1000	True	linear	488.01	11.224330000914051
2	uniform	1000	True	binary	16.88	0.6672449944744585
3	uniform	1000	True	interpolation	16.55	0.55411500397895
4	uniform	1000	False	linear	1000	24.685634998604655
5	uniform	1000	False	binary	18.94	0.6894899979670299
6	uniform	1000	False	interpolation	9.74	0.34735000554064754
7	uniform	3000	True	linear	1483.685	37.82085499551613
8	uniform	3000	True	binary	19.69	0.8200350021070335
9	uniform	3000	True	interpolation	16.625	0.6004750048305141
10	uniform	3000	False	linear	3000	78.58695999857446
11	uniform	3000	False	binary	22.64	0.846820003062021
12	uniform	3000	False	interpolation	9.515	0.3269299941166537
13	uniform	10000	True	linear	5024.47	136.04904499516124
14	uniform	10000	True	binary	23.35	0.9193400001095142
15	uniform	10000	True	interpolation	15.95	0.5886199960514205
16	uniform	10000	False	linear	10000	285.874804994819
17	uniform	10000	False	binary	26.38	1.063645004251157
18	uniform	10000	False	interpolation	9.085	0.35141500120516866
19	uniform	30000	True	linear	15429.975	445.5568100001983
20	uniform	30000	True	binary	26.49	1.0798899984365562
21	uniform	30000	True	interpolation	17.175	0.6269599998631747
22	uniform	30000	False	linear	30000	826.3433950014587
23	uniform	30000	False	binary	28.91	1.1514450034155743
24	uniform	30000	False	interpolation	10.7	0.3973399998358218
25	uniform	100000	True	linear	48033.825	1377.8039249973517
26	uniform	100000	True	binary	30.23	1.2525300007837359
27	uniform	100000	True	interpolation	18.35	0.6960099926800467
28	uniform	100000	False	linear	100000	3033.636350000961

Рисунок 12 - Таблица результатов измерения числа сравнений и времени поиска

```
CPU: Intel(R) Core(TM) i7-14650HX
Python: 3.13.5
Вариант: 16
Seed: 46
Размеры: [1000, 3000, 10000, 30000, 100000]
Повторы: 5
Ключей в серии: 200
Время: медиана времени серии / число ключей, микросекунды
```

Рисунок 13 - Характеристики компьютера и параметры эксперимента

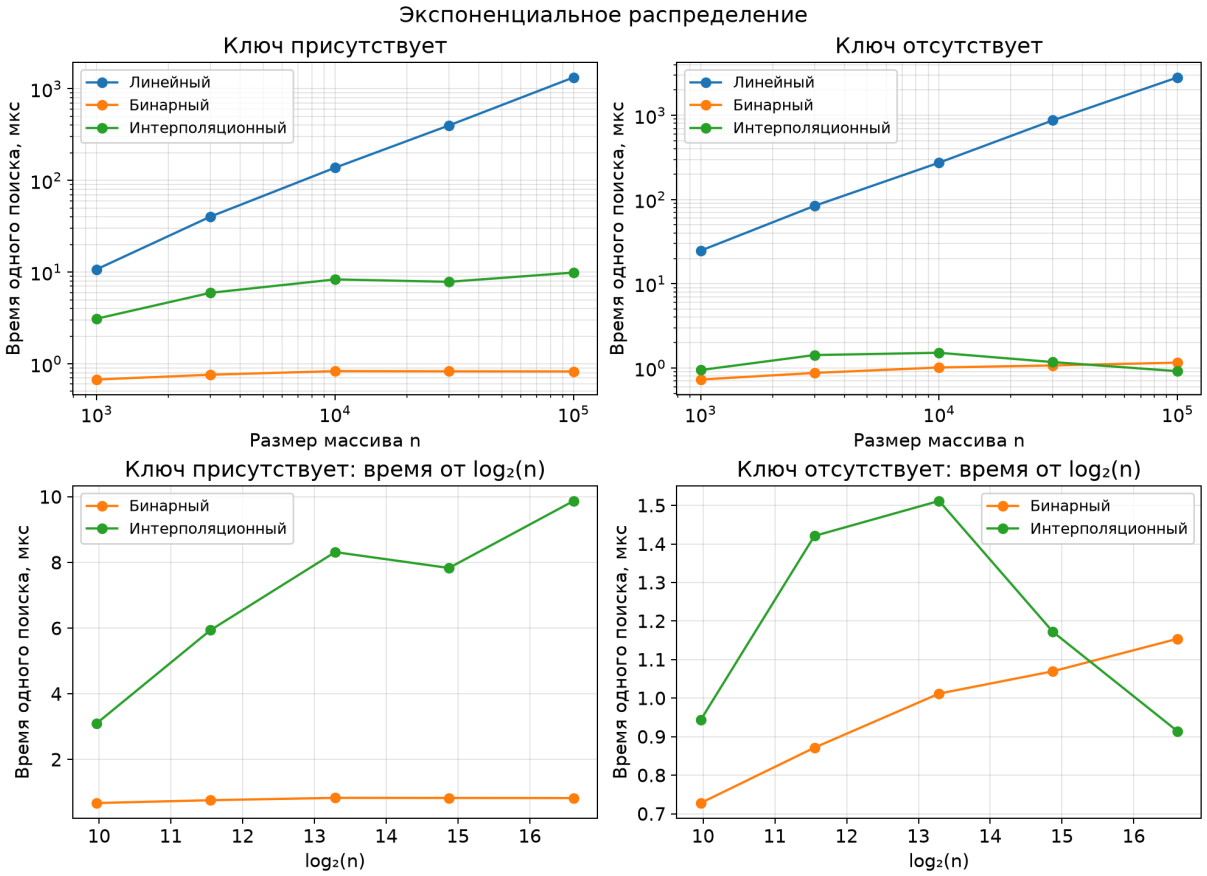


Рисунок 14 - Время поиска в массивах экспоненциального распределения

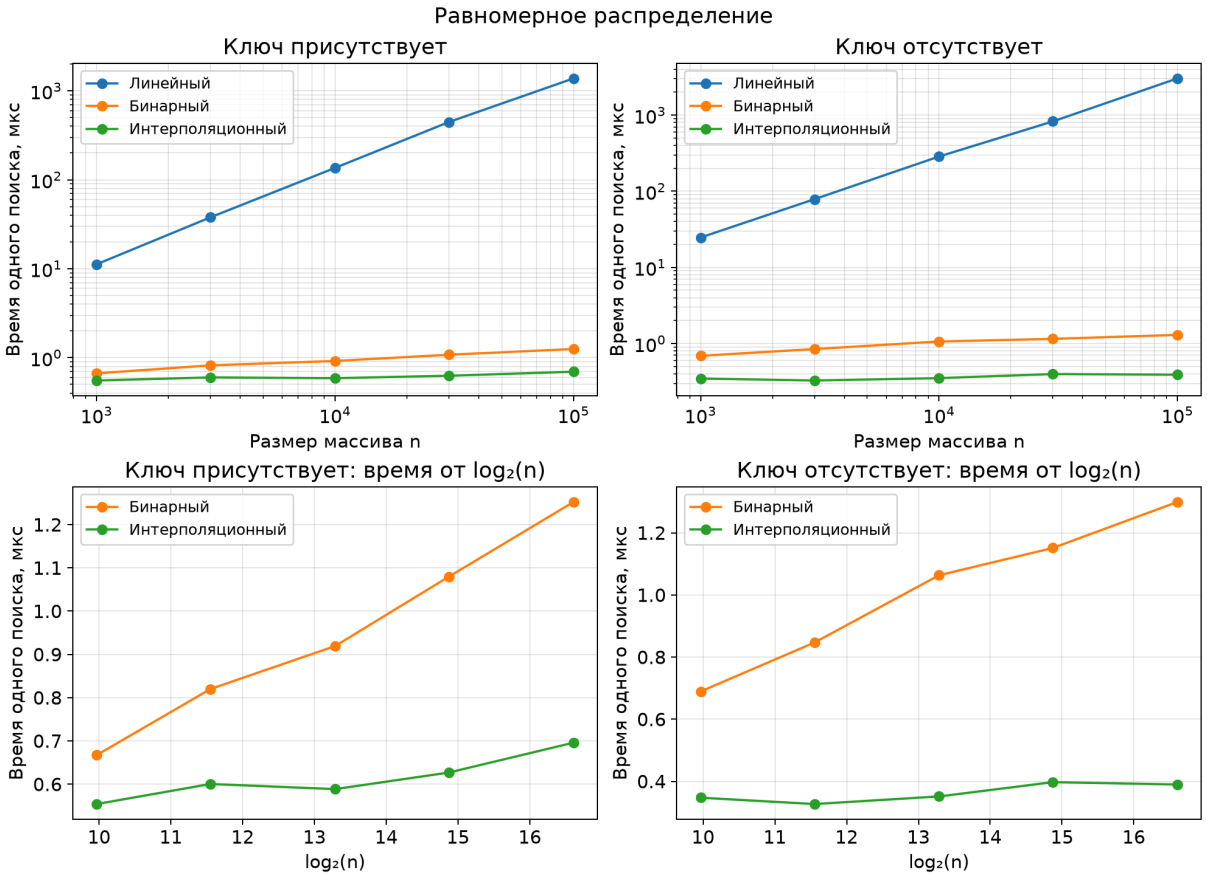


Рисунок 15 - Время поиска в массивах равномерного распределения

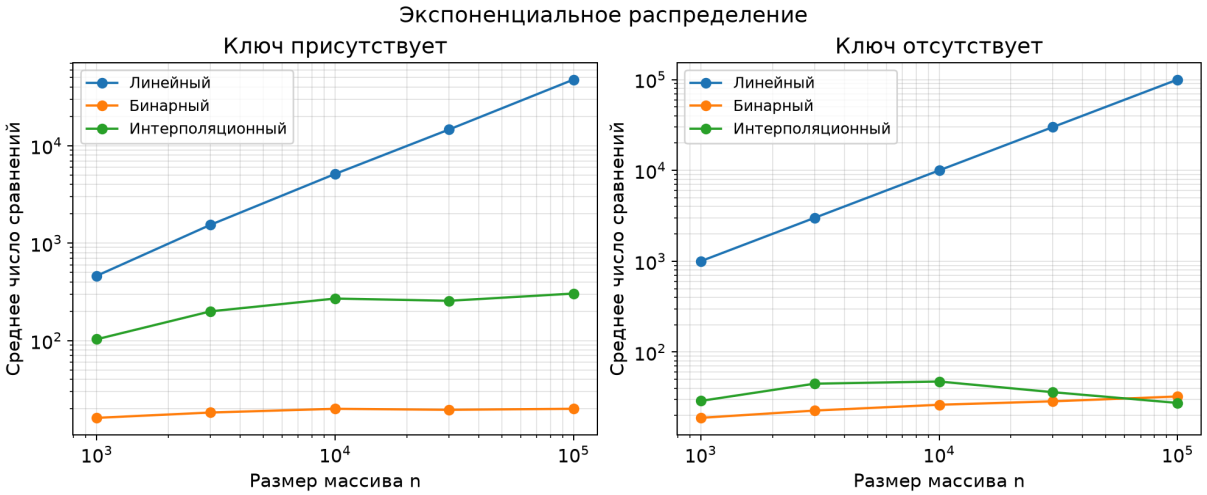


Рисунок 16 - Число сравнений при поиске в массивах экспоненциального распределения

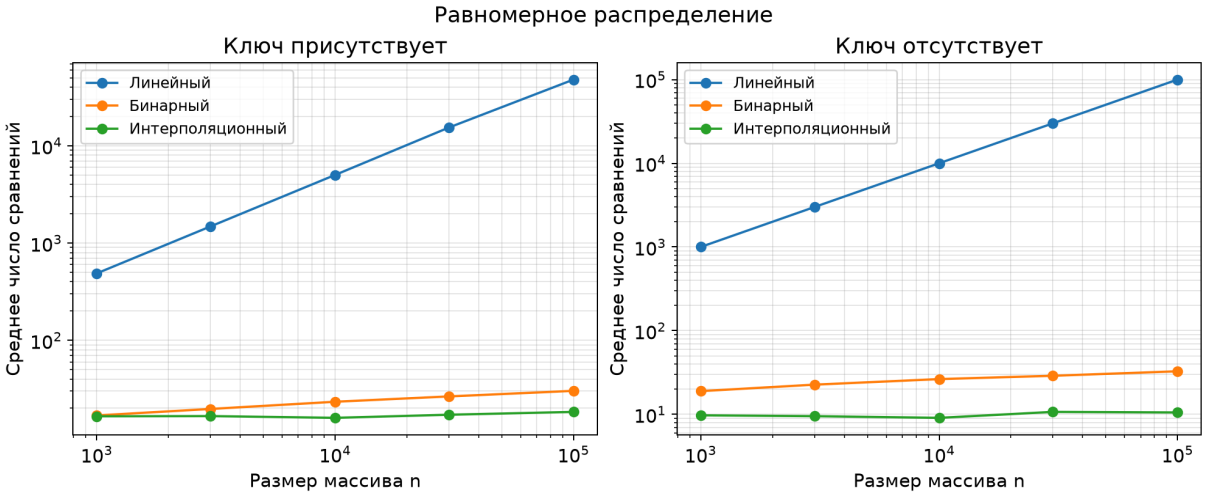


Рисунок 17 - Число сравнений при поиске в массивах равномерного распределения

Вывод: Получены зависимости числа сравнений и времени поиска от размера массива для трёх алгоритмов. Сохранены результаты для обоих распределений и двух видов ключей. Полученные таблица и графики позволяют сравнить эффективность алгоритмов в разных условиях.

4. Задание 4

Выполнена проверка трёх алгоритмов поиска с использованием оператора in, метода list.index() и функции bisect_left(). Каждый алгоритм проверен на 16 граничных и специальных случаях и 300 случайных массивах с seed=46. Проверены найденные и отсутствующие ключи, повторы и отрицательные значения. Дополнительно проверены счётчики сравнений на шести контрольных случаях и сохранность исходных массивов.

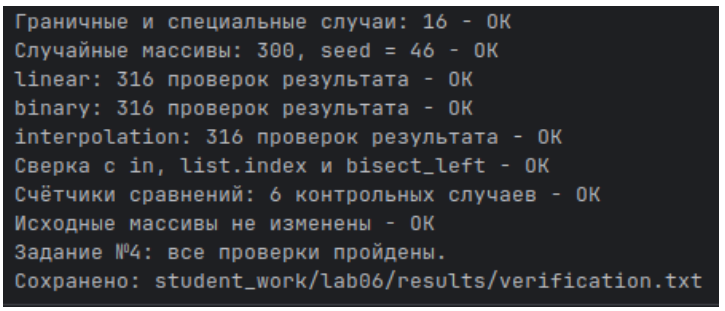


Рисунок 18 - Результаты верификации алгоритмов поиска

Вывод: Результаты трёх алгоритмов совпали с эталонами на всех проверенных входах. Линейный поиск возвращает первое вхождение, бинарный и интерполяционный возвращают допустимый индекс найденного элемента. Проверки счётчиков сравнений пройдены.

5. Задание 5

Проанализированы результаты измерения числа сравнений и времени бинарного и интерполяционного поиска. Рассмотрено влияние распределения данных и наличия искомого ключа. Полученные результаты сопоставлены с теоретическими оценками сложности.

При размере массива n=100000 получены следующие результаты. В таблице указаны среднее число сравнений и время одного поиска в микросекундах.

Распределение	Ключ	Сравнения, бинарный	Сравнения, интерполяционный	Время, бинарный, мкс	Время, интерполяционный, мкс
Равномерное	Присутствует	30,23	18,35	1,253	0,696
Равномерное	Отсутствует	32,69	10,53	1,300	0,390
Экспоненциальное	Присутствует	19,90	303,45	0,824	9,874
Экспоненциальное	Отсутствует	32,41	27,52	1,154	0,914

Таблица 1 - Сравнение бинарного и интерполяционного поиска при n=100000

На равномерных данных интерполяционный поиск оказался быстрее бинарного: при n=100000 примерно в 1,8 раза для присутствующего ключа и в 3,3 раза для отсутствующего. Равномерное распределение позволяет точнее предсказывать положение элемента. При увеличении n с 1000 до 100000 среднее число сравнений интерполяционного поиска изменилось с 16,55 до 18,35, бинарного с 16,88 до 30,23. Это согласуется со средней оценкой O(log log n) для интерполяционного поиска и

оценкой O(log n) для бинарного. На экспоненциальных данных интерполяционная формула хуже определяет позицию элемента. При n=100000 поиск присутствующего ключа потребовал 303,45 сравнения против 19,9 у бинарного и оказался примерно в 12 раз медленнее. Для отсутствующего ключа на том же размере интерполяционный поиск оказался немного быстрее. Этому способствует раннее исключение ключей, находящихся вне диапазона массива. Бинарный поиск сохраняет оценку O(log n) независимо от распределения. Интерполяционный поиск в худшем случае может достигать O(n), однако полученные замеры сами по себе не доказывают линейный рост.

Вывод: В эксперименте интерполяционный поиск показал преимущество на равномерных данных, а бинарный оказался устойчивее к неравномерному распределению. Результат также зависит от выбранных поисковых ключей.

Заключение

В ходе лабораторной работы реализованы и проверены линейный, бинарный и интерполяционный поиск в отсортированных массивах. Экспериментально исследованы число сравнений и время поиска при равномерном и экспоненциальном распределениях данных. Определено преимущество интерполяционного поиска на равномерных данных и устойчивость бинарного поиска к неравномерному распределению. Цель работы достигнута.